

ACTIVE-IA

Devolución de Trabajo Práctico

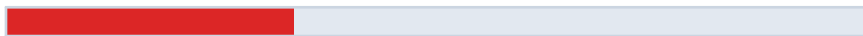
Materia:	PROG3-M2026 - Programación 3
Comisión:	COMI-15 (2026)
Trabajo:	Trabajo Práctico Integrador: Programación Funcional
Alumno:	Macarena Marinoni

Calificación Final: 84/100

EVALUACIÓN POR CRITERIOS

C1: Desarrollo de Clases y Relaciones (UML)

5/15



Las clases, herencia, atributos y enumeraciones están correctamente implementadas. Sin embargo, se ha utilizado 'List<DetallePedido>' en la clase Pedido en lugar de 'Set<DetallePedido>', lo cual es una violación directa del requisito de 'Uso estricto de colecciones tipo Set' especificado en la rúbrica.

C2: Implementación de Requerimientos con Programación Funcional

60/65



Todos los requerimientos prácticos (cálculo de total, productos disponibles, cantidad de ítems, stock crítico) han sido implementados de manera excelente utilizando exclusivamente Streams y programación funcional. Los métodos adicionales como 'findDetallePedidoByProducto' y 'deleteDetallePedidoByProducto' también demuestran un uso correcto de Streams.

C3: Comprensión y Aplicación de Conceptos de Programación Funcional

19/20



El código es muy limpio y claro, haciendo un uso correcto de operaciones intermedias (filter, mapToDouble, mapToInt) y terminales (sum, forEach, findFirst). La aplicación de lambdas y referencias a métodos es consistente y adecuada, demostrando una sólida comprensión de los conceptos de programación funcional.

FORTALEZAS

- Uso extensivo y correcto de la API de Streams de Java para resolver todos los requerimientos funcionales.
- Código limpio y bien estructurado, con una clara separación de responsabilidades entre las entidades.
- Aplicación adecuada de Lombok para reducir el boilerplate, mejorando la legibilidad del código.
- Manejo correcto de Optional en operaciones de búsqueda, como en 'findDetallePedidoByProducto'.

RECOMENDACIONES

1. Asegurarse de cumplir estrictamente con todos los requisitos de diseño, como el uso de 'Set' en lugar de 'List' para colecciones cuando se especifica.
2. Considerar hacer que el campo 'total' en la clase Pedido sea un getter calculado en lugar de un atributo almacenado, para garantizar que siempre refleje el estado actual de los detalles del pedido.
3. Implementar pruebas unitarias para los métodos funcionales, especialmente para 'calcularTotal' y los métodos de búsqueda/eliminación de detalles, para asegurar su robustez.
4. Explorar el uso de operaciones más avanzadas de Streams como 'groupingBy' o 'reducing' para futuros requerimientos complejos.

COMENTARIOS DEL EVALUADOR

Macarena ha demostrado un excelente dominio de la programación funcional con Java Streams, resolviendo la mayoría de los requerimientos de manera efectiva y con código claro. La estructura general del proyecto es buena y el uso de Lombok es un acierto. La principal área de mejora es la adherencia estricta a las especificaciones de diseño, particularmente en la elección de tipos de colecciones.